EXPLORATORY DATA ANALYSIS

Chapter 17

Investigating Unusual Observations

From a statistical flag to a defensible analytical decision

Course focus: rigorous quantitative and visual EDA with Python, NumPy, Pandas, Matplotlib's object-oriented API, and Seaborn.

This chapter follows Chapter 16 — Outlier and Anomaly Detection. It concentrates on investigation, explanation, documentation, and responsible decisions—not automatic deletion.

Learning objectives

• Distinguish an unusual observation from an error, an anomaly, and an influential point.

• Build an investigation workflow that starts with a flag but does not assume the flag is correct.

• Trace observations to source records, collection processes, units, timestamps, and business rules.

• Compare an observation with appropriate peers and quantify how much it changes summaries.

• Recognize masking, swamping, subgroup effects, and the danger of investigating values without context.

• Choose among retaining, correcting, excluding, or separately analyzing an observation—and document the rationale.

• Implement a reusable, auditable investigation table in Pandas.

> **Central rule:** an outlier detector identifies a candidate. Investigation establishes what the candidate represents. A statistical threshold is not a data-quality policy.

1. Where Chapter 17 fits

Chapter 16 answered: *Which observations look unusual under a chosen detection method?* This chapter asks: *Why are they unusual, are they valid, and what should an analyst do next?* The distinction matters because the same extreme value can be a typo, a rare legitimate event, a new operating regime, a measurement artifact, or an important signal.

Stage	Question	Output
Detection (Ch. 16)	Which records deserve attention?	Flags, scores, candidate list
Investigation (Ch. 17)	What explains each candidate?	Evidence, context, cause hypothesis
Decision	How should analysis treat it?	Retain, correct, exclude, or segment
Communication	Can another analyst reproduce the decision?	Audit trail and sensitivity results

2. Review of earlier chapters (Chapters 1–14)

The following ideas are prerequisites for investigating unusual observations. The immediately previous chapter, Chapter 16, is reviewed separately in the next section.

• **Ch. 1 — EDA:** investigation is iterative; description is not causal explanation.

• **Ch. 2 — Dataset understanding:** identify observational unit, grain, variable meaning, units, and data-generating process.

• **Ch. 3 — Data quality:** inspect validity, completeness, consistency, uniqueness, accuracy, and source reliability.

• **Ch. 4 — Numerical variables:** mean, median, spread, quartiles, IQR, and distribution shape respond differently to extremes.

• **Ch. 5 — Categorical variables:** rare categories may be valid; check labels, cardinality, denominators, and category meaning.

• **Ch. 6 — Date/time:** time zones, parsing, temporal gaps, event order, and collection windows can create apparent extremes.

• **Ch. 7 — Descriptive statistics:** robust and non-robust summaries answer different questions.

• **Ch. 8 — Distributions:** inspect tails, gaps, multimodality, heaping, and bounded or zero-inflated data.

• **Ch. 9 — Shape:** skewness, kurtosis, and quantiles describe distributional features; none proves an observation is erroneous.

• **Ch. 10 — Distribution plots:** histograms, ECDFs, boxplots, violin plots, and strip plots reveal different aspects.

• **Ch. 11 — Relationships:** a point unusual in a scatterplot may be ordinary conditionally or within a subgroup.

• **Ch. 12 — Multivariate visualization:** facets, heatmaps, and encodings help reveal combinations of variables.

• **Ch. 13 — Association:** correlation and association measures do not establish causation.

• **Ch. 14 — Dependence:** marginal distributions can hide conditional structure, heteroscedasticity, and nonlinear dependence.

3. Review of Chapter 16 — Outlier and Anomaly Detection

Chapter 16 introduced candidate detection methods: IQR fences, z-scores, modified z-scores based on the median absolute deviation (MAD), Mahalanobis distance, local density methods such as LOF, and Isolation Forest. It also distinguished global, contextual, and collective anomalies.

A flag depends on the selected variables, reference population, method, scale, and threshold. A value can be flagged in the full dataset but not within its correct peer group. Conversely, a value can look ordinary on one variable and unusual in a multivariate combination.

> **Expected reasoning for the Chapter 16 assignment/project:** a defensible solution should state the detection method, variables, reference population, threshold, and number of flagged records. It should retain the original rows, create explicit flag/score columns, and inspect examples rather than deleting flagged observations. If no assignment was submitted, this is a reasoning guide, not a grading result.

4. A disciplined investigation workflow

Step	Action	Evidence to collect
1. Preserve	Keep the raw record and stable identifier unchanged.	Source row ID, source file, ingestion batch
2. Reproduce	Recompute the flag with recorded settings.	Method, variables, threshold, software/version
3. Contextualize	Find the correct peer group and time window.	Entity, product, location, season, operating state
4. Validate	Check units, ranges, keys, timestamps, and source.	Data dictionary, raw event, system logs
5. Compare	Compare with peers and related variables.	Quantiles, rates, conditional plots, neighboring events
6. Test influence	Recompute conclusions with and without the candidate.	Effect on mean, SD, slope, rate, ranking
7. Decide	Choose a treatment based on evidence and purpose.	Decision, rationale, owner, uncertainty
8. Document	Make the decision reproducible.	Audit log, code, date, reviewer

> Do not begin by asking whether the point should be removed. Begin by asking what process could have produced it. This avoids treating a convenient statistical rule as an explanation.

5. Identify the observational unit and the peer group

A value is only unusual relative to a reference. For example, a daily electricity use of 900 kWh may be extraordinary for a small home but routine for a factory. A transaction of 50,000 currency units may be ordinary for a wholesale account but exceptional for a small retail customer.

Peer groups should be selected using variables that explain legitimate differences, such as product type, customer segment, machine, location, shift, age band, or season. Avoid defining peer groups using the outcome itself in a way that hides the anomaly.

Investigate at several levels: the whole dataset, the relevant subgroup, the entity's own history, and the immediate temporal or operational context.

```
# Example: compare an order with its product-category peers.
# Keep the original observations; do not delete flagged rows.
df["category_median"] = (
    df.groupby("category", dropna=False)["order_value"]
      .transform("median")
)
df["category_q1"] = (
    df.groupby("category", dropna=False)["order_value"]
      .transform(lambda s: s.quantile(0.25))
)
df["category_q3"] = (
    df.groupby("category", dropna=False)["order_value"]
      .transform(lambda s: s.quantile(0.75))
)
df["category_iqr"] = df["category_q3"] - df["category_q1"]
df["above_category_fence"] = (
    df["order_value"] >
    df["category_q3"] + 1.5 * df["category_iqr"]
)
```

6. Verify the record before interpreting it

6.1 Source and lineage

Trace the candidate back to the closest available source: original form, sensor message, transaction log, uploaded spreadsheet, or upstream database. Check whether the value changed during parsing, joining, aggregation, currency conversion, or unit conversion. A correct-looking value can still be attached to the wrong entity.

6.2 Units, scale, and decimal conventions

Common problems include centimetres mixed with metres, kilograms mixed with pounds, seconds mixed with milliseconds, local currency mixed with another currency, and decimal separators interpreted incorrectly. Confirm units from metadata and source systems; do not infer units from what seems plausible.

6.3 Keys, joins, and duplication

An unexpectedly large total may be caused by a many-to-many join or duplicated transaction, not an extreme underlying event. Check key uniqueness before and after joins. Compare row counts, distinct IDs, and aggregation totals.

```
# Basic lineage and integrity checks
print("Rows:", len(df))
print("Unique record IDs:", df["record_id"].nunique())
print("Duplicate IDs:", df["record_id"].duplicated().sum())

# Inspect a flagged record without mutating the source data.
candidate = df.loc[df["is_flagged"]].copy()
candidate = candidate.sort_values("order_value", ascending=False)

# Check whether IDs are duplicated in the raw table.
duplicate_records = df.loc[
    df["record_id"].duplicated(keep=False)
].sort_values("record_id")
```

6.4 Timestamps and event sequence

Check time zone, daylight-saving handling, clock drift, late-arriving events, and whether timestamps represent event time or ingestion time. For entity-level time series, sort within entity before calculating previous/next event gaps. A negative duration can indicate ordering or time-zone errors rather than a real anomaly.

6.5 Domain constraints and cross-field logic

Use explicit rules: quantity cannot be negative if the domain forbids it; an end time should not precede a start time; a cancelled order should not be treated as a completed sale; a person's date of birth cannot be after the event date. Rules must be grounded in the domain, not invented after seeing a surprising value.

7. Compare with relevant observations

A useful comparison is not always the global mean. Choose a comparison that matches the mechanism: same product, same customer, same machine, same hour, same season, or the same entity's recent history. Report sample

size and denominators; a subgroup of three records cannot support the same confidence as a subgroup of thousands.

```
# Build a peer comparison table for one candidate.
candidate_row = df.loc[df["record_id"].eq(candidate_id)].iloc[0]
peer_mask = (
    df["category"].eq(candidate_row["category"])
    & df["region"].eq(candidate_row["region"])
)
peers = df.loc[peer_mask, "order_value"].dropna()

comparison = {
    "candidate": candidate_row["order_value"],
    "peer_count": peers.size,
    "peer_median": peers.median(),
    "peer_q1": peers.quantile(0.25),
    "peer_q3": peers.quantile(0.75),
    "peer_max": peers.max(),
}
comparison
```

Useful questions include: Is the candidate extreme relative to peers? Is it extreme for this entity's own history? Do related variables support the same story? Does the pattern recur? Are there nearby observations that suggest a regime change?

8. Quantify influence on the conclusions

An extreme observation may be valid but influential. Influence is question-dependent: a record can materially change the mean while barely changing the median, or alter a fitted relationship while having little effect on a marginal distribution.

8.1 Compare summaries with and without the candidate

The purpose is sensitivity analysis, not quietly deleting inconvenient data. Always show both versions and explain why the exclusion is being considered. For a rate, recompute both numerator and denominator consistently.

```
values = df["order_value"].dropna()
without_candidate = values.loc[df.loc[values.index, "record_id"]
                               .ne(candidate_id)]

sensitivity = {
    "n_all": values.size,
    "mean_all": values.mean(),
    "median_all": values.median(),
    "std_all": values.std(),
    "n_without": without_candidate.size,
    "mean_without": without_candidate.mean(),
    "median_without": without_candidate.median(),
    "std_without": without_candidate.std(),
}
sensitivity
```

Interpret differences in the context of the analysis. A changed mean does not prove the record is wrong; it shows that the mean is sensitive to that record. If the business question concerns total revenue, a legitimate large transaction may be central, not disposable.

8.2 Influence on relationships

For a relationship, compare scatterplots and association measures with and without the candidate, but do not rely on a single correlation coefficient. Inspect whether the point changes slope, direction, or apparent form. Regression influence diagnostics such as leverage and Cook's distance belong to model diagnostics; they can be introduced as a boundary concept, but they do not replace EDA investigation.

8.3 Masking and swamping

Masking occurs when multiple unusual observations make one another appear less unusual. **Swamping** occurs when valid observations are incorrectly flagged because other observations distort the reference distribution. Inspect clusters of flags together, and consider robust or subgroup-aware summaries.

9. Visual investigation

Use complementary views. A histogram shows marginal distribution; an ECDF shows the candidate's percentile position; a boxplot supports peer comparison; a scatterplot shows relationships; a time plot reveals sequence and regime changes.

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

sns.histplot(data=df, x="order_value", ax=axes[0])
axes[0].set_title("Overall distribution")

sns.boxplot(
    data=df, x="category", y="order_value",
    ax=axes[1], showfliers=True
)
axes[1].tick_params(axis="x", rotation=45)
axes[1].set_title("Comparison by category")

sns.scatterplot(
    data=df, x="quantity", y="order_value",
    hue="is_flagged", ax=axes[2]
)
axes[2].set_title("Relationship and flagged records")

fig.tight_layout()
```

Matplotlib's object-oriented interface is used here: the figure and axes are explicit. In real work, avoid plotting every category if there are too many; select meaningful groups or use facets. Keep units and scales visible, and do not let color alone carry the message.

10. Classify the explanation

Classification	Typical evidence	Possible response
Data-entry or transcription error	Source record contradicts stored value; misplaced decimal or digit	Correct from authoritative source; preserve audit trail
Measurement / sensor error	Calibration issue, impossible reading, known instrument fault	Correct or mark invalid using documented rules
Unit / conversion error	Mixed units or faulty conversion pipeline	Standardize with verified conversion; revalidate affected batch
Join / duplication artifact	Repeated keys, inflated row count, duplicated event	Repair data pipeline; rebuild affected analysis
Legitimate extreme event	Source verified; domain context supports event	Retain; consider robust summaries and separate description
New regime / structural change	Persistent shift across later observations	Segment periods; investigate process change
Unknown / unresolved	Insufficient evidence or source unavailable	Retain with uncertainty flag; disclose limitation

11. Decide what to do — without automatic deletion

The decision must reflect both evidence and analytical purpose. A value that is invalid for one metric may still be useful as an indicator of a data pipeline problem. An extreme but valid event may need to remain in totals and be separately described.

Action	When defensible	Minimum safeguards
Retain	Value is valid and relevant to the target population.	Record why it is unusual; assess robust summaries.
Correct	An authoritative source establishes the correct value.	Preserve raw value, corrected value, source, and rule.
Exclude from a specific analysis	Value is invalid for that analysis or outside a predeclared scope.	State scope, rule, count, and sensitivity of results.
Analyze separately	A distinct regime or meaningful subgroup exists.	Define group transparently; avoid hiding inconvenient cases.
Mark unresolved	Evidence is insufficient to classify it.	Keep record; disclose uncertainty and revisit if evidence arrives.

> **Important:** do not overwrite the raw value. Prefer separate fields such as `raw_value`, `validated_value`, `investigation_status`, and `decision_reason`.

12. Build an investigation log

A professional EDA workflow should make every decision reviewable. A compact investigation log can be stored as a DataFrame or exported alongside the report. It should link to the original record using a stable identifier.

```
investigation_columns = [
    "record_id",
    "detector",
    "detector_score",
    "flag_reason",
    "peer_definition",
    "source_checked",
    "evidence",
    "classification",
    "decision",
    "decision_reason",
    "reviewer",
    "reviewed_at",
]

investigation_log = pd.DataFrame(columns=investigation_columns)

# Example row; replace with evidence from a real investigation.
new_case = {
    "record_id": "ORD-1042",
    "detector": "category_IQR",
    "detector_score": None,
    "flag_reason": "Above category-specific upper fence",
    "peer_definition": "category + region",
    "source_checked": "Pending",
    "evidence": "Not yet investigated",
    "classification": "Unresolved",
    "decision": "Retain pending review",
    "decision_reason": "No authoritative evidence for correction",
    "reviewer": "Analyst",
    "reviewed_at": pd.Timestamp.now(tz="UTC"),
}
investigation_log = pd.concat(
    [investigation_log, pd.DataFrame([new_case])],
    ignore_index=True
)
```

In production, use controlled status values, validate required fields, store timestamps consistently, and avoid putting sensitive personal information into free-text notes. Keep a versioned copy of the analysis code and the source-data reference.

13. Reusable investigation summary

```
def investigate_numeric_candidate(
    data, value_col, candidate_index, peer_mask=None
):
    """Return descriptive comparisons; this does not decide validity."""
    values = data[value_col].dropna()
    candidate_value = data.loc[candidate_index, value_col]

    if peer_mask is None:
        peers = values
        peer_label = "all non-missing observations"
    else:
        peers = data.loc[peer_mask, value_col].dropna()
        peer_label = "user-defined peer group"

    result = {
        "candidate_index": candidate_index,
        "candidate_value": candidate_value,
        "peer_definition": peer_label,
        "peer_n": int(peers.size),
        "peer_median": peers.median(),
        "peer_q1": peers.quantile(0.25),
        "peer_q3": peers.quantile(0.75),
        "peer_min": peers.min(),
        "peer_max": peers.max(),
    }

    if peers.size:
        result["candidate_percentile"] = (
            peers.le(candidate_value).mean() * 100
        )
    else:
        result["candidate_percentile"] = float("nan")

    return result

# Example:
# result = investigate_numeric_candidate(
#     df, "order_value", candidate_index=42,
#     peer_mask=df["category"].eq("Electronics")
```

```
    # )
```

This helper provides context, not a verdict. Percentile rank can be coarse when the peer group is small or contains many ties. For production use, define how missing values, duplicate indices, and candidate rows outside the peer group are handled.

14. Common investigation mistakes

• Deleting every value outside 1.5 × IQR without checking the domain.

• Comparing different populations as if they were exchangeable.

• Treating a high detector score as proof of a data error.

• Using a global reference when strong subgroup or temporal structure exists.

• Assuming that a valid value cannot be influential—or that an influential value must be invalid.

• Correcting a value based on intuition instead of an authoritative source.

• Changing the raw data in place and losing the original observation.

• Reporting only results after exclusion and hiding the sensitivity analysis.

• Failing to record the detector, threshold, peer definition, evidence, and decision.

• Confusing a rare event with a bad record; rarity is not the same as invalidity.

15. Summary

• Detection produces candidates; investigation establishes context and evidence.

• Start with the observational unit, source lineage, peer group, and data-generating process.

• Verify units, keys, joins, timestamps, domain rules, and related fields.

• Compare candidates with appropriate peers and the entity's own history.

• Measure influence on the actual conclusions, with and without the candidate.

• Classify the explanation and choose a purpose-specific response.

• Preserve raw data and document every correction, exclusion, or unresolved case.

16. Assignment

Part A — Conceptual reasoning

• Explain why an observation can be statistically unusual but valid.

• Give one example each of a transcription error, measurement error, legitimate extreme event, and structural change.

• Explain why a global threshold may be inappropriate when product categories have different scales.

• Distinguish masking from swamping.

• Explain why sensitivity analysis is not the same as justifying deletion.

Part B — Numerical investigation

Consider the values: 18, 19, 20, 20, 21, 22, 23, 24, 25, 96. The value 96 is flagged by an IQR-based rule.

• Calculate the mean, median, Q1, Q3, and IQR using Pandas' default quantile convention.

• Compare mean, median, and standard deviation with and without 96.

• List at least four plausible explanations for 96 and the evidence needed to distinguish them.

• Explain why the numbers alone cannot establish whether 96 should be removed.

Part C — Investigation design

A delivery dataset contains an order with a delivery duration of 9,800 minutes. Design a step-by-step investigation. Include at least three possible peer definitions, source checks, timestamp checks, related variables, and a decision log entry. State what you would do if the original source record is unavailable.

Part D — Reflection

Describe a situation in which retaining an extreme value is more honest and useful than removing it.

17. Coding project — Unusual Observation Investigation Module

Extend your cumulative EDA master notebook. Do not create a disconnected notebook unless you need a temporary scratchpad.

Project objective: build a reusable investigation workflow that takes flagged records from Chapter 16 and creates a transparent evidence table for each candidate.

• Load a dataset with a stable record identifier and meaningful domain variables.

• Import or recreate Chapter 16 flags without dropping any rows.

• Allow the analyst to define peer groups (for example, category, region, entity, or time period).

• Generate candidate-level context: peer count, median, quartiles, range, percentile, and related fields.

• Add source, unit, duplicate-key, timestamp, and cross-column validation checks where applicable.

• Create at least two complementary plots: a distribution view and a relationship or time-context view.

• Compute sensitivity summaries with and without selected candidates.

• Create an investigation log with classification, decision, rationale, reviewer, and review timestamp.

• Write a short report explaining at least three investigated candidates and clearly label unresolved cases.

Acceptance criteria: raw records remain intact; flags are reproducible; peer definitions are explicit; no observation is automatically deleted; decisions are supported by evidence; and another analyst can follow the audit trail.

Suggested deliverables: updated master notebook, cleaned investigation log, figures, and a concise Markdown report. For a client-facing EDA gig, present the investigation findings and limitations without claiming that every unusual point is an error.

18. Preview of Next Chapter

Chapter 18 — High-Dimensional EDA

The next chapter moves from a handful of variables to datasets with many columns. We will investigate dimensionality, redundancy, variable groups, missingness structure, correlation patterns, and the limits of pairwise analysis. We will use carefully chosen summaries and visualizations to understand high-dimensional data without confusing EDA with model-based dimensionality reduction.

Boundary: Chapter 18 remains an EDA chapter. It will help us understand many variables and their relationships; it will not become a feature-selection or model-training chapter.